

milestone4

March 28, 2026

1 Milestone 4 — VLM Membership Inference Attack

```
[1]: import sys
import os

# Ensure SAVE_DIR is defined before using it for sys.path.insert
from google.colab import drive
drive.mount('/content/drive')
SAVE_DIR = '/content/drive/MyDrive/milestone4/'

sys.path.insert(0, SAVE_DIR)

print(f'sys.path after insert: {sys.path}')
if os.path.exists(SAVE_DIR):
    print(f'Contents of {SAVE_DIR}: {os.listdir(SAVE_DIR)}')
else:
    print(f'{SAVE_DIR} does not exist.')
```

```
Mounted at /content/drive
sys.path after insert: ['/content/drive/MyDrive/milestone4/', '/content',
'/env/python', '/usr/lib/python312.zip', '/usr/lib/python3.12',
'/usr/lib/python3.12/lib-dynload', '', '/usr/local/lib/python3.12/dist-
packages', '/usr/lib/python3/dist-packages', '/usr/local/lib/python3.12/dist-
packages/IPython/extensions', '/root/.ipython']
Contents of /content/drive/MyDrive/milestone4/: ['classifier.pkl', 'train.csv',
'val.csv', 'test.csv', 'config.py', '.DS_Store', 'milestone4 (1).ipynb',
'batches.py', 'classifier.py', 'features.py', 'models.py', '__pycache__']
```

```
[2]: !pip install -q xgboost
!pip install num2words
```

```
Collecting num2words
  Downloading num2words-0.5.14-py3-none-any.whl.metadata (13 kB)
Collecting docopt>=0.6.2 (from num2words)
  Downloading docopt-0.6.2.tar.gz (25 kB)
  Preparing metadata (setup.py) ... done
Downloading num2words-0.5.14-py3-none-any.whl (163 kB)
163.5/163.5 kB
```

18.1 MB/s eta 0:00:00

Building wheels for collected packages: docopt

Building wheel for docopt (setup.py) ... done

Created wheel for docopt: filename=docopt-0.6.2-py2.py3-none-any.whl
size=13706

sha256=d2a73d5490a383bb6d64605c65c51c20de05c4d60dfb2684c7e720903a36cb20

Stored in directory: /root/.cache/pip/wheels/1a/bf/a1/4cee4f7678c68c5875ca89ea
ccf460593539805c3906722228

Successfully built docopt

Installing collected packages: docopt, num2words

Successfully installed docopt-0.6.2 num2words-0.5.14

```
[3]: import os
import pickle
import warnings
warnings.filterwarnings('ignore')
import torch

from models import load_data, load_model_and_processor, load_base_model_and_processor
from batches import process_dataset_in_batches
from classifier import build_membership_classifier, generate_submission

os.makedirs(SAVE_DIR, exist_ok=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

# Always load splits - needed for labels and test IDs even when using cached features.
splits = load_data()
```

Device: cuda

Loading dataset: UBC-SLIME/colx585_group_project_data

train.parquet: 0%| | 0.00/334M [00:00<?, ?B/s]

dev.parquet: 0%| | 0.00/67.0M [00:00<?, ?B/s]

test.parquet: 0%| | 0.00/338M [00:00<?, ?B/s]

Generating train split: 0%| | 0/6000 [00:00<?, ? examples/s]

Generating validation split: 0%| | 0/1200 [00:00<?, ? examples/s]

Generating test split: 0%| | 0/6000 [00:00<?, ? examples/s]

Train: 6000 samples (1500 members)

Validation: 1200 samples (300 members)

Test: 6000 samples

1.1 1. Extract Features

Run the cells in this section to extract features from scratch. Skip to **Section 2** if you already have cached CSVs on Drive.

```
[4]: # Load fine-tuned (target) and base (reference) models
processor, model = load_model_and_processor(device)
base_processor, base_model = load_base_model_and_processor(device)
```

Using dtype: torch.bfloat16

processor_config.json: 0.00B [00:00, ?B/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

chat_template.jinja: 0%| | 0.00/403 [00:00<?, ?B/s]

config.json: 0.00B [00:00, ?B/s]

tokenizer_config.json: 0%| | 0.00/838 [00:00<?, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

Loading model: UBC-SLIME/colx_585_vlm

model.safetensors: 0%| | 0.00/513M [00:00<?, ?B/s]

Loading weights: 0%| | 0/471 [00:00<?, ?it/s]

generation_config.json: 0%| | 0.00/135 [00:00<?, ?B/s]

Model loaded.

Using dtype: torch.bfloat16

processor_config.json: 0%| | 0.00/68.0 [00:00<?, ?B/s]

chat_template.json: 0%| | 0.00/424 [00:00<?, ?B/s]

preprocessor_config.json: 0%| | 0.00/486 [00:00<?, ?B/s]

config.json: 0.00B [00:00, ?B/s]

tokenizer_config.json: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

added_tokens.json: 0%| | 0.00/92.0 [00:00<?, ?B/s]

special_tokens_map.json: 0.00B [00:00, ?B/s]

Loading model: HuggingFaceTB/SmolVLM-Base

You are using a model of type idfics3 to instantiate a model of type smolvlm. This is not supported for all configurations of models and can yield errors.

model.safetensors: 0%| | 0.00/4.49G [00:00<?, ?B/s]

```
Loading weights: 0%|          | 0/657 [00:00<?, ?it/s]
generation_config.json: 0%|          | 0.00/132 [00:00<?, ?B/s]
Model loaded.
```

```
[5]: train_features = process_dataset_in_batches(
      model, processor, splits['train'], has_labels=True, batch_size=200,
      base_model=base_model, base_processor=base_processor
    )
    train_features.to_csv(f'{SAVE_DIR}train.csv', index=False)
    print('Train features saved.')
```

Processing 6000 samples in batches of 200...

```
Batch 1/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:09<00:00, 1.54it/s]
Batch 2/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 3/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 4/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.59it/s]
Batch 5/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 6/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 7/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 8/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 9/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.59it/s]
Batch 10/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.59it/s]
Batch 11/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.60it/s]
Batch 12/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
```

Batch 13/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 14/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 15/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.59it/s]

Batch 16/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.59it/s]

Batch 17/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.58it/s]

Batch 18/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 19/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 20/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 21/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 22/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 23/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 24/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.59it/s]

Batch 25/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 26/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 27/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 28/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

```

Batch 29/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.60it/s]

Batch 30/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]

Done. 6000 samples processed.
Train features saved.

```

```

[6]: val_features = process_dataset_in_batches(
    model, processor, splits['validation'], has_labels=True, batch_size=200,
    base_model=base_model, base_processor=base_processor
)
val_features.to_csv(f'{SAVE_DIR}val.csv', index=False)
print('Val features saved.')

```

```

Processing 1200 samples in batches of 200...
Batch 1/6 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.58it/s]

Batch 2/6 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]

Batch 3/6 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.58it/s]

Batch 4/6 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]

Batch 5/6 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.59it/s]

Batch 6/6 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]

Done. 1200 samples processed.
Val features saved.

```

```

[7]: test_features = process_dataset_in_batches(
    model, processor, splits['test'], has_labels=False, batch_size=200,
    base_model=base_model, base_processor=base_processor
)
test_features.to_csv(f'{SAVE_DIR}test.csv', index=False)
print('Test features saved.')

```

```

Processing 6000 samples in batches of 200...
Batch 1/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]

```

Batch 2/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 3/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 4/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.59it/s]

Batch 5/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 6/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.59it/s]

Batch 7/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 8/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.59it/s]

Batch 9/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 10/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.59it/s]

Batch 11/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 12/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 13/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 14/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.60it/s]

Batch 15/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 16/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:05<00:00, 1.59it/s]

Batch 17/30 (200 samples)
 Extracting features: 100%| | 200/200 [02:06<00:00, 1.58it/s]

```

Batch 18/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 19/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.60it/s]
Batch 20/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.59it/s]
Batch 21/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:05<00:00, 1.60it/s]
Batch 22/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.59it/s]
Batch 23/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.58it/s]
Batch 24/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:07<00:00, 1.57it/s]
Batch 25/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:07<00:00, 1.57it/s]
Batch 26/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:07<00:00, 1.57it/s]
Batch 27/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.59it/s]
Batch 28/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.58it/s]
Batch 29/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.58it/s]
Batch 30/30 (200 samples)
Extracting features: 100%|      | 200/200 [02:06<00:00, 1.58it/s]
Done. 6000 samples processed.
Test features saved.

```

1.2 2. Load Cached Features

Run this cell instead of Section 1 when resuming a session.

```
[8]: # import pandas as pd
      # train_features = pd.read_csv(f'{SAVE_DIR}train.csv')
```

```
# val_features = pd.read_csv(f'{SAVE_DIR}val.csv')
# test_features = pd.read_csv(f'{SAVE_DIR}test.csv')
# print('Features loaded.')
```

1.3 3. Train Classifier & Evaluate

```
[9]: results = build_membership_classifier(train_features, val_features)

print('\n Summary ')
for name, r in results.items():
    print(f'{name:25s} AUC={r["auc"]:.4f} TPR@FPR=0.1={r["tpr_at_fpr01"]:.4f}')

best_name = max(results, key=lambda x: results[x]['auc'])
best = results[best_name]
print(f'\nBest: {best_name} AUC={best["auc"]:.4f}')
```

Features: 25 | Train: 6000 | Val: 1200

Individual feature AUCs (top 10):

caption_loss_diff	AUC=0.7299
perplexity_ratio	AUC=0.7299
ref_min_k10_prob	AUC=0.6879
ref_caption_loss	AUC=0.6813
ref_caption_perplexity	AUC=0.6813
loss_ratio	AUC=0.6793
ref_min_k20_prob	AUC=0.6730
ref_min_token_prob	AUC=0.6620
mean_prob_diff	AUC=0.6175
mean_entropy_diff	AUC=0.6109

Training Logistic Regression...

AUC=0.7760 TPR@FPR=0.1=0.2800

Training XGBoost...

AUC=0.7793 TPR@FPR=0.1=0.2833

Top-5: ['perplexity_ratio', 'caption_loss_diff', 'ref_min_k10_prob', 'ref_min_token_prob', 'ref_caption_perplexity']

Building ensemble (simple average)...

AUC=0.7790 TPR@FPR=0.1=0.2600

Summary

Logistic Regression	AUC=0.7760	TPR@FPR=0.1=0.2800
XGBoost	AUC=0.7793	TPR@FPR=0.1=0.2833
Ensemble	AUC=0.7790	TPR@FPR=0.1=0.2600

Best: XGBoost AUC=0.7793

Summary		
Logistic Regression	AUC=0.7760	TPR@FPR=0.1=0.2800
XGBoost	AUC=0.7793	TPR@FPR=0.1=0.2833
Ensemble	AUC=0.7790	TPR@FPR=0.1=0.2600

Best: XGBoost AUC=0.7793

1.4 4. Generate Submission

```
[10]: classifier_path = f'{SAVE_DIR}classifier.pkl'
with open(classifier_path, 'wb') as f:
    pickle.dump(best, f)
print(f'Classifier saved to {classifier_path}')

submission = generate_submission(
    splits['test'], test_features, best,
    output_path='submission.csv'
)
print(submission.head())
```

Classifier saved to /content/drive/MyDrive/milestone4/classifier.pkl

Submission saved to submission.csv (6000 rows)

Score range: [0.0300, 0.8201]

	id	is_member
0	cauldron_ln_8044	0.466525
1	cauldron_ln_7863	0.735952
2	mix_SI_UT_5400_19	0.104288
3	mix_SI_UT_5400_896	0.449061
4	cauldron_ln_17842	0.091573

```
[11]: from google.colab import files
files.download('submission.csv')
```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>